Authentication

1. Feature Overview

Staff and portal authentication via username/password, JWT bearer tokens, profile retrieval, password change, and logout (audit). Roles + `ACCESS_*` authorities loaded from DB (union of assigned roles).

Status: Implemented

2. Business Purpose

Protect banking APIs and the Angular **staff** and **customer portal** apps. Support lockout after failed logins and authority-based access.

3. User Workflow

1. Open `/` login page
2. Enter credentials (optional remember-me)
3. Receive JWT; staff → `/app/dashboard`, portal-only → `/portal/accounts`
4. Profile at `/app/profile`; change password at `/app/profile/password`
5. Logout calls `POST /auth/logout` then clears session
6. 401 from API → interceptor clears session → login

4. Execution Flow

See CALL_FLOW.md and SEQUENCE_DIAGRAMS.md. Login success/failure and logout write `audit_event` (AUTH).

5. Database Tables

`users`, `roles`, `user_roles`, `role_access` (lock/failed-attempt fields on `users`; optional `customer_id` for portal).

6. REST APIs

Method	Path	Auth
POST	`/auth/login`	Public
POST	`/auth/logout`	Authenticated
GET	`/auth/me`	Authenticated
PUT	`/auth/password`	Authenticated

7. Controllers

- `com.bankone.auth.controller.AuthenticationController`

- `com.bankone.auth.controller.TestController` (`GET /api/hello`)

8. Services

- `AuthenticationService` — `login`, `getCurrentUserProfile`, `changePassword`
- `JwtService` — generate / validate / extract
- `LoginAttemptService` — `incrementFailedAttempts`
- `CustomUserDetailsService` — `loadUserByUsername`

9. Repositories

- `UserRepository`, `UserRoleRepository` (user package)
- `RoleRepository` (role package)

10. DTOs

`LoginRequest`, `LoginResponse`, `UserProfileResponse`, `ChangePasswordRequest`

11. Entities

`User`, `Role`, `UserRole` (+ `BankUserDetails` adapter)

12. Utility Classes

None dedicated; JWT via JJWT library.

13. Configuration Classes

- `com.bankone.common.config.SecurityConfig`
- `jwt.secret`, `jwt.expiration` in `application.properties`
- Frontend: `api-config.ts`, `auth.interceptor.ts`, `auth-guard.ts`

14. Validation Rules

- Change password: `@NotBlank`, new password `@Size(min=8)`, confirm must match (service-level)

15. Security Rules

- Stateless sessions; CSRF disabled
- Only `/auth/login` (and OPTIONS/error) permitAll among auth routes
- Roles not in JWT claims — refreshed from DB per request

16. Exception Handling

`JwtAuthenticationEntryPoint` (401), `JwtAccessDeniedHandler` (403), plus `GlobalExceptionHandler` for business errors.

17. Logging

Spring Security TRACE enabled in dev properties (noise in production).

18. Audit Events

No dedicated audit event table. User `lastLogin` / failed attempts updated on login path.

19. Testing Strategy

- Redeploy script smoke: `POST /auth/login` with `admin` / `Admin@123`
- Manual: wrong password → lockout behavior
- Guard: visit ADMIN route as EMPLOYEE

20. Future Extension Guide

- Embed roles in JWT (trade-off: stale roles vs DB hit)
- Refresh tokens
- MFA
- Move secrets to env / Liberty jasypt

Future Modification Guide

Requirement: Change JWT expiry

Item	Detail
Files	`BankOne/src/main/resources/application.properties`
Classes	`JwtService` (reads `@Value`)
Methods	`generateToken()` uses `jwt.expiration`
Impact	Active tokens keep old expiry until re-login; document for ops

Requirement: Change lockout threshold

Item	Detail
Files	`LoginAttemptService.java`, `User` entity fields, possibly `AuthenticationService.login()`
Methods	`incrementFailedAttempts()`, lock check in `BankUserDetails` / login
Impact	UX messaging on login page

Requirement: Add public endpoint

Item	Detail
Files	`SecurityConfig.java`
Methods	`securityFilterChain` → `authorizeHttpRequests`
Impact	Must not accidentally open `/accounts/**`

Call hierarchy (login)

```
AuthenticationController.login()
        ↓
AuthenticationService.login()
        ↓
AuthenticationManager.authenticate()
        ↓
CustomUserDetailsService.loadUserByUsername()
        ↓
JwtService.generateToken()
```